

Technical & Coding Interview Guide

For Senior Software Engineers — 8+ Years

Stack focus: Java, Spring, AWS, Kafka, Postgres, Oracle, React, Node

Conceptual depth questions, coding exercises, and code-review tasks

How to Use This Guide

This is structured for two rounds:

- **Round 1 — Technical (45–60 min):** rapid-fire depth questions across Java, Spring, JVM, databases, Kafka, AWS, and frontend.
- **Round 2 — Coding (60–75 min):** one warm-up algorithm, one applied coding task, and one code-review exercise.

What to look for at 8+ years

- They explain why, not just what. 'HashMap uses an array of buckets' is mid-level. 'It chains on collisions and switches to a tree above 8 entries per bucket' is senior.
- They've debugged the things they describe. Anyone can recite the GC algorithms; few can tell you what a long G1 mixed-collection pause looks like in production.
- Code is readable, defensive, and testable. Not necessarily clever.
- They know the limits of their knowledge. 'I'd have to check the exact behaviour' is fine — pretending is not.

Round 1 — Technical Questions

Use 8–12 of these depending on seniority signal. Each is 3–6 minutes. Follow up on any answer that sounds memorized.

Java Core & Concurrency

Q1. Walk me through what happens internally when you put an object into a HashMap. What changed in Java 8?

Model answer

HashMap stores entries in an internal array of buckets. On put, it computes the key's hashCode, applies a supplemental hash (to reduce clustering), and indexes into the array as $\text{hash} \& (\text{capacity} - 1)$. Capacity is always a power of two so the mask works.

If the bucket is empty, the entry goes there. If not, it walks the linked list of entries, comparing each with equals(). If a match is found, the value is replaced; otherwise the entry is appended.

Java 8 changed two important things:

- When a bucket's chain exceeds 8 entries (and the table is at least 64 in size), it converts that bucket from a linked list to a balanced tree (red-black). Lookups in a degenerate bucket go from $O(n)$ to $O(\log n)$. Mitigates hash-collision attacks.
- Rehashing on resize was simplified — entries no longer change relative order in a way that caused the famous Java 7 concurrent-resize infinite loop. (Still not thread-safe; just less catastrophic.)

Resize: when $\text{size} > \text{capacity} \times \text{loadFactor}$ (default 0.75), capacity doubles and entries are redistributed. Resize is $O(n)$.

Follow-up probes

- What goes wrong if hashCode and equals are inconsistent?
- HashMap vs ConcurrentHashMap vs Collections.synchronizedMap — when do you use which?
- What's the difference between LinkedHashMap and HashMap?

Q2. Explain the difference between synchronized, ReentrantLock, and StampedLock. When have you actually used something other than synchronized?

Model answer

- **synchronized**: intrinsic lock baked into every object. Simple, JVM-optimized (biased locking, lock coarsening, escape analysis). Always reentrant. Released automatically on exit/exception. Limitation: you can't try-lock, can't interrupt waiting threads, can't have multiple condition variables.
- **ReentrantLock**: explicit lock. Supports tryLock with timeout, interruptible acquisition, fairness (FIFO) option, and multiple Condition objects. Cost: you must remember to unlock in a finally block. I use it when I need tryLock with a timeout — e.g., wrapping an operation that should give up rather than wait indefinitely on contention.

- **ReentrantReadWriteLock:** separates read and write locks. Reads can be concurrent; writes are exclusive. Good for read-heavy data structures where ConcurrentHashMap doesn't fit.
- **StampedLock (Java 8):** adds optimistic reads. You get a 'stamp' and read without locking; afterward you validate the stamp. If a write happened in between, you fall back to a real read lock. Higher throughput than ReadWriteLock when writes are rare. Not reentrant — caveat.

In practice 95% of the time synchronized is fine and you let the JVM optimize it. I reach for ReentrantLock when I need tryLock semantics, and for StampedLock only in measured hot paths where contention is dominated by reads.

Follow-up probes

- What does 'fair' mean on a lock, and what's the trade-off?
- If two threads each hold a lock the other needs — deadlock. How do you detect it in production?

Q3. What's the difference between CompletableFuture, ExecutorService, and virtual threads (Project Loom)? When do you pick which?

Model answer

- **ExecutorService:** the foundation. A thread pool you submit Runnables/Callables to. Returns Future for the result. Future.get() blocks the calling thread. Good for fire-and-forget or limited parallelism, but composing async work is awkward.
- **CompletableFuture:** non-blocking composition on top of executors. You can chain (thenApply, thenCompose), combine (allOf, anyOf), and handle errors (exceptionally) without blocking. Default uses ForkJoinPool.commonPool() — almost always wrong for I/O; pass your own executor.
- **Virtual threads (Java 21+):** lightweight threads scheduled onto a small pool of carrier threads by the JVM. Blocking calls unmount the virtual thread instead of holding a platform thread. You write straight-line blocking code and get the scalability of async. Game-changer for I/O-bound services.

My current decision tree:

- Server endpoint doing I/O on Java 21+ → virtual threads (Tomcat/Jetty support it; Spring Boot 3.2+ has a flag).
- Pre-21, async composition of multiple downstream calls → CompletableFuture, with an explicit executor sized to the workload.
- CPU-bound parallelism → ExecutorService with a pool sized to CPU cores (or ForkJoinPool for divide-and-conquer).

Follow-up probes

- Why is ForkJoinPool.commonPool() dangerous for I/O work?
- What's 'pinning' with virtual threads, and when does it happen?
- CompletableFuture.thenApply vs thenCompose — when do you use each?

Q4. What is the volatile keyword? What does it guarantee, and what does it NOT guarantee?

Model answer

volatile guarantees two things: visibility (a write is immediately visible to other threads — no caching in CPU registers or thread-local memory) and ordering (the JVM and CPU won't reorder reads/writes of a volatile field across other operations — the happens-before relationship).

It does NOT guarantee atomicity for compound operations. A volatile int counter where two threads do counter++ will still race — because ++ is read, increment, write, and only each individual access is atomic.

Typical use: a stop flag (volatile boolean running) where one thread writes and others read. For counters use AtomicInteger; for more complex state use a lock or a concurrent collection.

Follow-up probes

- Double-checked locking — does it need volatile? Why?
- Is volatile cheaper or more expensive than synchronized?

Q5. Explain Java's memory model with respect to GC. What are G1's main phases? What's a typical sign of GC pressure in production?

Model answer

Heap is split into Young (Eden + two Survivor spaces) and Old generations. Most objects die young; the generational hypothesis is what makes GC efficient. Minor GC collects Young — fast, stop-the-world but short. Major/Full GC touches Old — slow, much longer pause.

G1 (the default since Java 9) divides the heap into equal-sized regions (~2k of them) and collects regions with the most garbage first. Phases:

- Young collection: pauses, evacuates Young regions to Survivor/Old.
- Concurrent marking: runs alongside the application, marks reachable objects in Old.
- Mixed collection: evacuates both Young and the most garbage-rich Old regions.
- Full GC: fallback when G1 can't keep up — STW, expensive. You don't want to see this.

Signs of GC pressure in production:

- p99 latency climbing while p50 is flat (pause times affect tail latency disproportionately).
- CPU spent in GC > 5% sustained.
- Frequent Full GCs in logs.
- Heap usage sawtoothing higher each cycle — memory leak.

First-line tools: enable GC logs (-Xlog:gc*), use JFR / Mission Control or async-profiler. For leaks, heap dump + Eclipse MAT.

Follow-up probes

- ZGC vs G1 vs Shenandoah — when would you switch?
- What's a 'humongous' allocation in G1 and why does it matter?

Q6. What's the difference between checked and unchecked exceptions? What's your team's policy and why?

Model answer

Checked exceptions extend `Exception` (but not `RuntimeException`) and must be declared or caught. Unchecked extend `RuntimeException` and don't have to be.

The original intent was that checked = recoverable, unchecked = programming error. In practice this hasn't aged well. Checked exceptions break functional interfaces (no checked exceptions in `Function.apply`), force boilerplate (`try/catch/rethrow` up the stack), and encourage the worst pattern in the language: catch and swallow.

My team's policy: use unchecked exceptions for almost everything. Define a small hierarchy of domain exceptions (`PaymentException`, `NotFoundException`) extending `RuntimeException`, and let them propagate to a top-level handler (`@ControllerAdvice` in Spring) that maps them to HTTP responses. Checked exceptions only where we want to force the caller to handle a specific recoverable case — which is rare.

Follow-up probes

- Where in the stack do you catch exceptions? Where do you let them propagate?
- How do you avoid losing the cause chain when wrapping?

Spring & Spring Boot

Q7. Walk me through how @Transactional works. What are the common pitfalls?

Model answer

@Transactional is implemented by a proxy. Spring wraps the bean in a CGLIB or JDK dynamic proxy; when you call an annotated method through the proxy, an aspect starts a transaction (gets a connection from the DataSource, calls setAutoCommit(false)), runs the method, and either commits or rolls back.

Pitfalls — every one of these I've seen in code review:

- **Self-invocation:** @Transactional methods called from within the same class bypass the proxy. The annotation is silently ignored. Fix: inject self, refactor, or use AspectJ load-time weaving.
- **Default rollback only on RuntimeException:** checked exceptions don't roll back unless you specify rollbackFor. People are surprised by this constantly.
- **Propagation confusion:** REQUIRED (default) joins an existing transaction. REQUIRED_NEW suspends and starts a new one — useful for audit writes that must commit even if the parent rolls back. NESTED uses savepoints. NEVER and MANDATORY are rare but useful.
- **Long-running transactions:** an @Transactional method that calls an external API holds a DB connection for the duration. The connection pool drains, the service slows. Don't put network I/O inside transactions.
- **Read-only flag:** @Transactional(readOnly = true) lets Hibernate skip dirty-checking and lets some drivers route to read replicas. Free perf win on read endpoints.
- **private/final methods:** AOP proxies can't intercept them. The annotation is ignored, no warning.

Follow-up probes

- What isolation level does Spring use by default, and what does it mean in Postgres?
- How would you implement 'try-three-times-then-fail' retry on a deadlock?

Q8. What's the difference between @Component, @Service, @Repository, and @Controller? Are they functionally identical?

Model answer

All four register the class as a Spring-managed bean. They're 99% interchangeable in behavior — but each carries semantic meaning and a couple of side effects:

- **@Component:** generic stereotype. No special behaviour.
- **@Service:** no extra behaviour; signals 'this is a business service.' Pure convention.
- **@Repository:** adds exception translation. Persistence-layer exceptions (SQLException, HibernateException) are converted to Spring's DataAccessException hierarchy. This is a real behaviour difference.
- **@Controller / @RestController:** registers the bean as an MVC handler. Spring scans these for @RequestMapping methods. @RestController = @Controller + @ResponseBody.

In practice, using the right stereotype is about readability and convention. Auto-configuration and component scanning don't care if it's `@Component` vs `@Service`.

Q9. Explain Spring Boot auto-configuration. How do you debug 'why is this bean configured / not configured'?

Model answer

Auto-configuration is built on conditional bean definitions. Spring Boot ships hundreds of `@Configuration` classes (in `spring-boot-autoconfigure`, third-party starters, etc.) each gated by conditions like `@ConditionalOnClass` (the class is on the classpath), `@ConditionalOnMissingBean` (no user-provided override), `@ConditionalOnProperty`, etc.

At startup, Spring evaluates each conditional config and applies the ones whose conditions match. So 'just having `spring-boot-starter-data-jpa` on the classpath' triggers JPA auto-config because the conditions evaluate true.

Debugging:

- Run with `--debug`. Spring prints the auto-configuration report — Positive matches (applied), Negative matches (skipped, with reason), Exclusions, and Unconditional classes.
- `/actuator/conditions` endpoint gives the same report at runtime.
- To override an auto-configured bean, define your own with `@Primary` or matching the same type — `@ConditionalOnMissingBean` will back off.
- To exclude auto-config entirely: `@SpringBootApplication(exclude = DataSourceAutoConfiguration.class)`.

Q10. How do you secure a Spring Boot REST API? Walk me through your default setup.

Model answer

Default setup for a service-to-service or user-facing API:

- Spring Security on the classpath, which enforces auth on all endpoints by default. Explicit `permitAll` on health checks and Swagger if applicable.
- JWT-based stateless auth via OAuth2 Resource Server. The JWT is validated against the issuer's JWKS (e.g., Cognito, Auth0, Keycloak). No sessions, no CSRF needed for stateless API.
- Method-level authorization with `@PreAuthorize` for fine-grained rules (e.g., `@PreAuthorize("hasRole('ADMIN') or #userId == authentication.name")`).
- Input validation with `@Valid` + JSR-380 annotations. Reject bad input before it reaches business code.
- CORS configured explicitly — never `permitAll()` origins in prod.
- HTTPS enforced at the load balancer; HSTS header set.
- Rate limiting at the API gateway (or with `Bucket4j` inside the service).
- Secrets in AWS Secrets Manager or Parameter Store, injected via Spring Cloud AWS — never in property files.
- Audit logging of every state-changing request (who, what, when) to a separate log stream.

What I avoid: rolling my own JWT validation, basic auth in production, returning stack traces in error responses.

Follow-up probes

- How do you handle JWT expiry / refresh on the client side?
- Session-based auth vs JWT — when is session better?

Postgres, Oracle & Data Access

Q11. Explain database isolation levels. What does Postgres' default give you, and what anomaly does it allow?

Model answer

Standard isolation levels, from weakest to strongest:

- **READ UNCOMMITTED:** can see other transactions' uncommitted writes (dirty reads). Postgres doesn't actually implement this — it treats it as READ COMMITTED.
- **READ COMMITTED:** only sees committed data, but each statement sees a fresh snapshot. Allows non-repeatable reads (same query twice returns different rows) and phantom reads. Postgres default.
- **REPEATABLE READ:** snapshot taken at the start of the transaction; all reads in the transaction see that snapshot. No non-repeatable or phantom reads. In Postgres this is implemented as snapshot isolation.
- **SERIALIZABLE:** the strongest. Transactions appear to run one after another. In Postgres, implemented as SSI (Serializable Snapshot Isolation) — optimistic; can abort transactions with a serialization failure that the app must retry.

The default READ COMMITTED is fine for most CRUD. I bump to REPEATABLE READ or SERIALIZABLE for: financial calculations, anything 'read-then-write' that depends on the read being stable, or batch jobs that produce reports.

Oracle is similar but historically defaults to READ COMMITTED with statement-level snapshots; SERIALIZABLE in Oracle is snapshot-based too.

Follow-up probes

- Walk me through the lost-update anomaly. How do you prevent it?
- Optimistic vs pessimistic locking — when do you pick which?
- What's SELECT FOR UPDATE used for, and what's the downside?

Q12. A query that ran in 50ms last week is taking 8 seconds today. The data hasn't grown much. Walk me through diagnosis.

Model answer

First: EXPLAIN (ANALYZE, BUFFERS) on the slow query. Compare actual vs estimated rows — if estimates are way off, the planner has bad statistics. Run ANALYZE on the involved tables.

Common causes for sudden-onset slowness without big data growth:

- **Plan flip:** Postgres planner switched from index scan to seq scan because statistics drifted. ANALYZE refreshes stats; auto-analyze thresholds may need tuning.
- **Bloat:** lots of updates/deletes on the table → dead tuples not vacuumed → index and table size bloated. Check pg_stat_user_tables.n_dead_tup. Manual VACUUM (FULL if desperate, but it locks).
- **Index no longer used:** someone dropped or invalidated an index, or the query now uses a parameter value that has poor selectivity (e.g., status='active' when 99% of rows are active).

- **Parameter sniffing / generic plan:** prepared statements can pick a generic plan that's bad for some bind values. Set `plan_cache_mode = force_custom_plan` to test.
- **Lock contention:** the query is waiting on a lock. Check `pg_stat_activity` and `pg_locks`.
- **New noisy neighbor:** shared RDS or some other workload eating I/O. Check CloudWatch / `pg_stat_io`.

Oracle has similar tooling: EXPLAIN PLAN, AWR reports, V\$SESSION for blocking, `dbms_stats.gather_table_stats` for stats.

Q13. When would you reach for a composite index? When does an index actually hurt you?

Model answer

Composite index on (a, b, c) is useful for queries that filter on a, or (a, b), or (a, b, c) — left-prefix rule. A query that only filters on b won't use it.

Reach for one when:

- A query consistently filters on multiple columns together.
- You can satisfy the query from the index alone (covering index) — Postgres' INCLUDE clause adds non-key columns for this.
- You're filtering AND sorting by overlapping columns: (status, created_at) lets a query for active rows in date order skip a sort.

Indexes hurt when:

- The table is write-heavy. Every insert/update touches every index. More indexes = slower writes.
- Storage cost — large indexes on large tables can exceed the table size.
- Bloat from many updates of indexed columns.
- The planner picks the wrong index. Sometimes fewer is better.
- Low cardinality: an index on a boolean column with 50/50 distribution is rarely useful — a seq scan is faster.

My habit: start with no indexes beyond primary keys and foreign keys. Add indexes driven by query plans on real workloads. Drop unused ones (`pg_stat_user_indexes` shows usage).

Kafka

Q14. Explain consumer groups, partitions, and the rebalance protocol. What's a problem rebalances cause in practice?

Model answer

A topic is split into partitions. Each partition is an ordered, append-only log. A consumer group is a set of consumers that collectively consume a topic, with each partition assigned to exactly one consumer in the group. Partitions are the unit of parallelism — you can't have more active consumers than partitions.

Rebalance happens when group membership changes: a consumer joins, leaves, or is considered dead (session timeout exceeded). The coordinator reassigns partitions across the remaining consumers.

Problem: during a rebalance, all consumers in the group stop consuming. With the old eager protocol, a single consumer joining stalls the entire group for the duration of the rebalance. For a group consuming a slow topic, this can mean seconds of stalled processing — and if your processing is slow enough that it triggers another rebalance, you can get stuck in a loop.

Mitigations:

- Cooperative/incremental rebalance protocol (default since Kafka 2.4) — only revokes the partitions that need to move.
- Tune `max.poll.interval.ms` higher if processing is slow; `max.poll.records` lower so you commit more often.
- Static membership (`group.instance.id`) so a restart doesn't trigger a rebalance — the consumer just resumes when it comes back.
- Keep processing fast or move heavy work to async — don't do 10-second downstream calls inside poll loops.

Follow-up probes

- How does Kafka decide which partition a message goes to?
- How do you ensure message ordering for a particular user across partitions?

Q15. How do you handle a poison message that always crashes the consumer?

Model answer

Bad pattern: consumer crashes → restarts → fetches the same offset → crashes again. Topic stalls. I've seen this take down production.

The right pattern is a DLQ workflow:

- Wrap message processing in try/catch.
- On retryable failure (network blip, downstream 5xx), retry in-process with backoff a few times, then publish to a retry topic with a delay.
- On non-retryable failure (parsing error, validation error, bug we can't recover from), publish to a DLQ topic with metadata: original payload, original offset/partition, exception trace, timestamp.
- Commit the offset after either successful processing OR successful DLQ publish. The poison message no longer blocks the partition.

- DLQ has alerts and tooling — humans inspect, fix the bug, and either replay messages to the main topic or write them off.

Spring Kafka has built-in support via `SeekToCurrentErrorHandler` / `DefaultErrorHandler` + `DeadLetterPublishingRecoverer`.

Key principle: never silently drop. Every poison message ends up somewhere a human can find it.

AWS

Q16. Walk me through deploying a Spring Boot service on AWS. Why ECS vs EKS vs Lambda?

Model answer

For most Spring Boot services I default to ECS on Fargate:

- No EC2 instances to manage.
- Tight AWS integration — ALB target groups, IAM task roles, CloudWatch, Secrets Manager.
- Simpler than EKS for teams that don't already run Kubernetes.
- Deployment via CodeDeploy or a CI pipeline (GitHub Actions / GitLab CI) building the image, pushing to ECR, updating the ECS service definition.

EKS when:

- The org already runs Kubernetes — consistency wins.
- You want portability across clouds or on-prem.
- You need Kubernetes-specific tooling (Istio, Argo, Helm) or your developers already know it.

Lambda when:

- Spiky or low-volume workload — cost wins over EC2/Fargate.
- Event-driven (S3 trigger, SQS consumer, API Gateway endpoint).
- Caveat for Spring Boot: cold start is brutal (3–10 seconds). Solutions: SnapStart (Java 11/17 on Corretto), Spring Native / GraalVM, or stick to lightweight handlers without Spring.

Networking: the service goes in private subnets, behind an internal or internet-facing ALB. The DB (RDS) sits in another private subnet. NAT gateway for outbound to third parties. WAF on the ALB for public APIs.

Q17. What's the difference between RDS, Aurora, and DynamoDB? When would you pick each?

Model answer

- **RDS:** managed Postgres / MySQL / Oracle on standard storage. AWS handles patching, backups, multi-AZ failover. You pick instance size and storage; you manage versions and parameters. Good for most workloads up to a few TB.
- **Aurora:** AWS's reimplementation of Postgres / MySQL storage on a distributed log-structured backend. Up to 128 TB, 6-way replication, faster failover, much faster read replicas (shared storage means replicas don't replay logs). Costs more per hour but much cheaper at high I/O. Aurora Serverless v2 scales capacity automatically — good for variable workloads.
- **DynamoDB:** managed NoSQL key-value/document store. Single-digit-millisecond reads/writes at any scale. Per-request or provisioned pricing. Strong eventual or consistent reads. Best for: well-known access patterns, very high scale, zero-ops requirement, or workloads where you'd otherwise need sharding.

My defaults: Aurora Postgres for new transactional services (RDS Postgres if budget tight). DynamoDB for session stores, user profiles, event logs, or anything where the access pattern is one or two key-lookups and we need scale. Oracle on RDS only when the org standardizes there.

Q18. How do you handle secrets and configuration in AWS for a Spring Boot service?

Model answer

Tiered approach:

- Static, non-sensitive config (timeouts, feature flags, log levels) → application.yml in the repo, environment-specific values via Spring profiles.
- Environment-specific non-sensitive values → SSM Parameter Store (Standard tier). Free, integrates with Spring Cloud AWS.
- Secrets (DB password, API keys, JWT signing keys) → Secrets Manager. Supports automatic rotation, audit trail, encryption at rest with KMS.
- DB credentials specifically → Secrets Manager with rotation enabled. The Lambda rotation function changes the password and updates the secret atomically; the app refetches on auth failure.

Access: IAM task role on the ECS task grants permission to read specific secrets and parameters. No keys in the container.

What I avoid: secrets in environment variables visible in the task definition, secrets in CloudWatch logs (sanitize), checking .env files into git, sharing one secret across multiple services.

React & Node

Q19. Explain the React rendering lifecycle and what `useMemo`, `useCallback`, and `React.memo` actually do. When are they premature optimization?

Model answer

React re-renders a component when its state or props change, or when its parent re-renders. The render produces a new virtual DOM; React diffs against the previous and applies only the changed parts to the real DOM.

- **React.memo:** wraps a component so it skips re-rendering if its props are referentially equal (shallow compare) to last time.
- **useMemo:** memoizes a computed value across renders. Recomputes only when dependencies change.
- **useCallback:** memoizes a function reference. Same function instance across renders unless dependencies change. Useful when you pass callbacks to memoized child components.

Premature when:

- The component is cheap to render. The memoization machinery (closure, deps comparison) can cost more than the render it saves.
- The dependency array changes every render anyway — you've just added overhead.
- You're memoizing for 'performance' without profiling. React DevTools profiler is the only honest source of truth.

My rule: write straightforward code first. Profile when something feels slow. Only then add memoization where it shows up in the profile.

React 19's compiler can do much of this automatically — worth watching if the team is on the latest.

Q20. When would you use SSR / Next.js vs a pure SPA? What are the trade-offs?

Model answer

Pure SPA (Vite + React, no server rendering):

- Best for: internal tools, dashboards, apps behind authentication where SEO doesn't matter.
- Pros: simplest mental model, deploy as static assets to S3 + CloudFront, no Node server to operate.
- Cons: poor SEO (search bots and link-preview crawlers don't run JS well), slower first paint.

Next.js (or Remix) with SSR / SSG:

- Best for: marketing sites, e-commerce, content-heavy public pages.
- Pros: better SEO (pre-rendered HTML), faster perceived load (content visible before JS hydrates), supports static export for cacheable pages.
- Cons: now you operate a Node server (or accept Vercel's hosting), more complex caching story, hydration mismatch bugs.

Hybrid: Next.js for the public marketing pages, a separate SPA for the authenticated app. Different concerns, different optimizations.

Q21. Walk me through the Node event loop. Why does CPU-bound work block it?

Model answer

Node is single-threaded for JavaScript execution. The event loop runs JS callbacks; I/O (file, network) is delegated to libuv's thread pool or to the OS, which calls back when ready.

Phases of the event loop (roughly): timers, pending callbacks, poll (where I/O callbacks run), check (setImmediate), close. Microtasks (Promise.then, queueMicrotask) drain between every phase.

Why CPU-bound blocks: while a JS callback is running, no other JS runs. A heavy synchronous loop (e.g., big JSON parse, crypto, image processing) freezes the entire process — no new requests are handled, timers don't fire, sockets back up.

Solutions:

- Move CPU work off the main thread: `worker_threads` for true parallelism, or `child_process` for isolation.
- Use streams for large payloads — process chunk by chunk instead of holding the whole thing in memory.
- Push genuinely heavy work out of Node entirely — to a Java service, a Lambda, or a queue worker.

Mental model: Node is fantastic for I/O orchestration, terrible for compute. Match the workload to the tool.

Round 2 — Coding Exercises

Three exercises. Pick one warm-up, one applied, and one code review. Total 60–75 minutes.

At this seniority level the goal isn't to see whether they can solve LeetCode-hard. It's to see how they think about real code: how they decompose a problem, what they test, how they handle edge cases, and whether their code would survive review.

Section A — Warm-up (10–15 minutes)

One of these. Goal: see that they can write working code under mild pressure.

QC1. Write a function that takes a list of integers and returns the top K most frequent elements.

Example

Input: `nums = [1, 1, 1, 2, 2, 3, 4, 4, 4, 4]`, `k = 2`
Output: `[4, 1]` // 4 appears 4 times, 1 appears 3 times

Model answer (Java)

```
public List<Integer> topKFrequent(int[] nums, int k) {
    Map<Integer, Integer> counts = new HashMap<>();
    for (int n : nums) {
        counts.merge(n, 1, Integer::sum);
    }

    // Min-heap of size k, ordered by frequency
    PriorityQueue<Map.Entry<Integer, Integer>> heap =
        new PriorityQueue<>(Comparator.comparingInt(Map.Entry::getValue));

    for (var entry : counts.entrySet()) {
        heap.offer(entry);
        if (heap.size() > k) {
            heap.poll(); // remove smallest
        }
    }

    List<Integer> result = new ArrayList<>(k);
    while (!heap.isEmpty()) {
        result.add(heap.poll().getKey());
    }
    Collections.reverse(result); // largest first
    return result;
}
```

Complexity

- Time: $O(n \log k)$ — n elements pushed/popped through a heap of size k .
- Space: $O(n)$ for the counts map.

What to watch for

- Do they use a heap of size k (good) or sort all n entries (also fine but $O(n \log n)$)?

- Bonus: bucket sort approach gets $O(n)$ — buckets indexed by frequency.
- Do they handle edge cases — empty input, k larger than distinct elements?
- Stream-based one-liner is fine and idiomatic; ask them to explain trade-offs.

QC2. Implement a thread-safe LRU cache with a fixed capacity. Methods: `get(key)` and `put(key, value)`, both $O(1)$.

Model answer (Java)

```
public class LRUCache<K, V> {
    private final int capacity;
    private final LinkedHashMap<K, V> map;
    private final ReentrantLock lock = new ReentrantLock();

    public LRUCache(int capacity) {
        this.capacity = capacity;
        // accessOrder = true → reorders on get(), enabling LRU eviction
        this.map = new LinkedHashMap<>(capacity, 0.75f, true) {
            @Override
            protected boolean removeEldestEntry(Map.Entry<K, V> eldest) {
                return size() > LRUCache.this.capacity;
            }
        };
    }

    public V get(K key) {
        lock.lock();
        try {
            return map.get(key); // also bumps access order
        } finally {
            lock.unlock();
        }
    }

    public void put(K key, V value) {
        lock.lock();
        try {
            map.put(key, value); // eldest evicted automatically if over capacity
        } finally {
            lock.unlock();
        }
    }
}
```

What to watch for

- They know `LinkedHashMap` has `accessOrder` + `removeEldestEntry` built in.
- If they implement it from scratch — `HashMap` + doubly linked list — that's also good. Senior candidates can do either.
- Thread safety: `ReentrantLock` or `synchronized` is fine. `ConcurrentHashMap` alone doesn't give LRU semantics.

- Push them: 'how would you do this for a multi-instance service?' — answer: Redis with allkeys-lru eviction policy.

Section B — Applied Coding (30–45 minutes)

One of these. Goal: see them build something realistic. They should design API + types + implementation + tests.

QC3. Build a rate limiter as a reusable component. Requirements:

- Configurable: N requests per window of T seconds, per key (e.g., per user).
- Thread-safe.
- Method: boolean tryAcquire(String key).
- Bonus: discuss how you'd adapt this for distributed use (multiple service instances).

Model answer — sliding window approach

```
public class SlidingWindowRateLimiter {
    private final int maxRequests;
    private final long windowMillis;
    private final Map<String, Deque<Long>> requests = new ConcurrentHashMap<>();

    public SlidingWindowRateLimiter(int maxRequests, Duration window) {
        this.maxRequests = maxRequests;
        this.windowMillis = window.toMillis();
    }

    public boolean tryAcquire(String key) {
        long now = System.currentTimeMillis();
        long cutoff = now - windowMillis;

        Deque<Long> timestamps = requests.computeIfAbsent(key, k -> new
ArrayDeque<>());

        synchronized (timestamps) {
            // drop expired entries from the front
            while (!timestamps.isEmpty() && timestamps.peekFirst() < cutoff) {
                timestamps.pollFirst();
            }
            if (timestamps.size() >= maxRequests) {
                return false;
            }
            timestamps.offerLast(now);
            return true;
        }
    }
}
```

Discussion points to push on

- **Memory growth:** the map grows forever as new keys arrive. Need eviction — scheduled cleanup or expiring map (Caffeine with `expireAfterAccess`).
- **Token bucket alternative:** smoother burst handling. Refill rate + bucket capacity. `Bucket4j` is the Java library that does this well.
- **Sliding-log vs sliding-window-counter vs fixed-window:** trade-off between accuracy and memory.

- **Distributed:** Redis with INCR + EXPIRE for fixed window, or a Lua script for atomic sliding window. The library spring-cloud-gateway-rate-limiter does this. Trade-off: every request makes a Redis round-trip.
- **Failure mode:** if Redis is down, do we fail open (allow all) or fail closed (deny all)? Usually fail open with an alert — better to serve traffic than to outage yourself.

Tests they should write

- Allow up to N requests in window.
- Reject the N+1th.
- After window passes, accept again.
- Different keys are independent.
- Concurrent calls don't over-allow (run with multiple threads).

QC4. Build a small REST endpoint in Spring Boot:

- POST /orders — accepts { customerId, items: [{ productId, quantity }] }, returns the created order with an id and total.
- GET /orders/{id} — returns the order.
- Validation, error handling, and at least two unit tests.
- In-memory storage is fine — focus is on structure, not persistence.

What strong code looks like

They should structure this as: Controller (thin, request/response mapping) → Service (business logic, validation) → Repository (storage). DTOs separate from domain entities. Bean Validation annotations on the request DTO. @ControllerAdvice for error mapping. Tests for the service layer with mock repository, plus at least one @WebMvcTest for the controller.

Skeleton — Controller

```
@RestController
@RequestMapping("/orders")
@RequiredArgsConstructor
public class OrderController {
    private final OrderService orderService;

    @PostMapping
    public ResponseEntity<OrderResponse> create(
        @Valid @RequestBody CreateOrderRequest request) {
        Order order = orderService.create(request);
        return ResponseEntity
            .created(URI.create("/orders/" + order.getId()))
            .body(OrderResponse.from(order));
    }

    @GetMapping("/{id}")
    public OrderResponse get(@PathVariable String id) {
        return OrderResponse.from(orderService.findById(id));
    }
}
```

Skeleton — DTOs with validation

```
public record CreateOrderRequest(
    @NotBlank String customerId,
    @NotEmpty @Valid List<OrderItemRequest> items) {}

public record OrderItemRequest(
    @NotBlank String productId,
    @Positive int quantity) {}
```

Skeleton — Global error handling

```
@RestControllerAdvice
public class GlobalExceptionHandler {
    @ExceptionHandler(NotFoundException.class)
    ResponseEntity<ErrorResponse> notFound(NotFoundException e) {
        return ResponseEntity.status(404).body(new ErrorResponse(e.getMessage()));
    }

    @ExceptionHandler(MethodArgumentNotValidException.class)
    ResponseEntity<ErrorResponse> validation(MethodArgumentNotValidException e) {
        String msg = e.getBindingResult().getFieldErrors().stream()
            .map(f -> f.getField() + ": " + f.getDefaultMessage())
            .collect(Collectors.joining(", "));
        return ResponseEntity.status(400).body(new ErrorResponse(msg));
    }
}
```

What to watch for

- Do they separate DTOs from entities, or pass the JPA entity straight through the controller? The latter leaks the domain into the API.
- Do they validate input or trust it?
- Do they handle the 'order not found' case? Or let a NoSuchElementException bubble up as a 500?
- Test quality: do tests describe behaviour or just exercise methods? @WebMvcTest for the slice, plain JUnit for the service.
- Do they think about concurrency on the in-memory store? ConcurrentHashMap or a lock around the create-then-store sequence?

Section C — Code Review (15–20 minutes)

Show them the code below. Ask: 'this came in as a PR — walk me through your review.' Stop talking after that and let them work.

This is one of the most signal-rich exercises at the senior level. It reveals what they notice.

The PR

```
@RestController
public class UserController {

    @Autowired
    UserRepository userRepository;

    @Autowired
    EmailService emailService;

    @GetMapping("/users/{id}")
    public User getUser(@PathVariable String id) {
        return userRepository.findById(id).get();
    }

    @PostMapping("/users")
    @Transactional
    public User createUser(@RequestBody Map<String, Object> body) {
        User user = new User();
        user.setEmail((String) body.get("email"));
        user.setName((String) body.get("name"));
        user.setPassword((String) body.get("password"));
        userRepository.save(user);

        emailService.sendWelcome(user.getEmail());

        return user;
    }

    @GetMapping("/users")
    public List<User> listUsers() {
        List<User> all = userRepository.findAll();
        List<User> active = new ArrayList<>();
        for (User u : all) {
            if (u.getStatus().equals("ACTIVE")) {
                active.add(u);
            }
        }
        return active;
    }
}
```

Issues a strong reviewer will flag (roughly in priority order)

- **Password handling:** stored in plain text from the request — no hashing, no validation, returned in the response. Critical security bug.

- **Returning entity directly:** User leaks internal fields (password, internal flags) to the API consumer. Need a DTO.
- **Untyped Map<String, Object>:** no validation, no compile-time safety, no API contract. Use a request DTO with @Valid.
- **findById(id).get():** throws NoSuchElementException on missing — leaks as a 500. Should be 404.
- **Email-send inside @Transactional:** the email may send and then the DB roll back. Worse, the transaction holds a DB connection while waiting on SMTP. Move email send to after commit (TransactionSynchronization.afterCommit, or publish an event).
- **Field injection (@Autowired on field):** harder to test, hides dependencies, can mask circular dependencies. Use constructor injection (or @RequiredArgsConstructor).
- **listUsers does findAll + in-memory filter:** fine for 10 users, catastrophic for 10 million. Push the filter to the database (findByStatus). Add pagination.
- **No pagination on listUsers:** any unbounded list endpoint is an outage waiting to happen.
- **No input validation:** email format, name length, password strength — all missing.
- **No idempotency on POST:** double-clicks create duplicate users. Idempotency key or a unique constraint on email with a clear 409 response.
- **No authorization:** anyone can read/list any user. Where's @PreAuthorize or the auth check?
- **Logging:** no logging at all. At minimum, log user creation (without password).
- **Return type on createUser:** returns 200 OK with body. Should be 201 Created with a Location header.
- **equals on String literal:** u.getStatus().equals("ACTIVE") — null-pointer if status is null. "ACTIVE".equals(u.getStatus()) is safer. Better, an enum.

Calibration

- **Strong senior:** spots 8+ of these including the password issue, the email-in-transaction, and the unbounded list.
- **Average senior:** 5–7, definitely the password and DTO issues. May miss email-in-transaction.
- **Below bar:** fewer than 5, or misses the security issues. The password storage and the entity leakage are non-negotiable at 8+ years.

Closing — Evaluating the Candidate

Technical round rubric

- **Depth:** do they answer the 'why,' not just the 'what'?
- **Breadth:** comfortable across multiple parts of the stack — not just Java?
- **Practical experience:** answers grounded in real incidents, not just theory?
- **Calibration:** do they know what they don't know?

Coding round rubric

- **Clarification:** did they ask about requirements, edge cases, scale before coding?
- **Structure:** is the code well-organized, with clear separation of concerns?
- **Correctness:** does it actually work? Edge cases handled?
- **Testing:** did they write or describe tests? Test for behaviour, not implementation?
- **Code quality:** would this pass code review at your company?
- **Communication:** did they think aloud, explain trade-offs, respond to feedback?

Common no-hire signals at 8+ years

- Can't write working code for a basic problem without hand-holding.
- In the code review, misses the security issues.
- Uses framework features without knowing what they do underneath (cargo-cult Spring).
- Defensive when challenged — argues instead of investigating.
- Has been a senior for 5 years and still can't explain @Transactional pitfalls or how a HashMap works.
- Treats tests as an afterthought ('I'd add tests later').

Common strong-hire signals

- Asks clarifying questions before coding.
- Writes small, runnable code first, then iterates.
- Mentions tests early and explains what they'd test.
- Spots security issues in the code review without prompting.
- Can switch context — Java, then SQL, then a bit of React — without obvious gaps.
- Has opinions but holds them lightly. 'I usually do X, but I can see Y here.'